Otros mercados y otros timeframes

AlgoProject — manual de uso. Actualizado el 2026-10-07 con `tools/manual.py`.

1. 30. Paso 10 de una estrategia — ¿viaja la lógica de ESTA estrategia?
2. 5. Paso 10 — ¿la lógica viaja a otros mercados?
3. 39. Paso 10 de la población entera — ¿viaja la lógica?, un veredicto por estrategia
4. 31. crossTF — ¿el edge aguanta si lo miras en un reloj más lento?

30. Paso 10 de una estrategia — ¿viaja la lógica de ESTA estrategia?

> La idea y cómo se juzga: `07-otros-mercados-y-timeframes.pdf › 05-retest-mercados`. Esta página es el comando de una sola estrategia y lo que enseña su pestaña en la ventana.

Qué pregunta responde

Lo mismo que el paso 10, pero mirando **una** estrategia en profundidad: sus nueve hechos, mercado por mercado, año por año, para entender por qué salió Pasa, Limbo o Falla.

Cuándo lo usas, y cuándo no

Cuando el lote (`07-otros-mercados-y-timeframes.pdf › 39-crossmarket-lote`) dejó una estrategia en Limbo o Falla y quieres ver **dónde** se rompe, o antes de decidir si una superviviente merece seguir. No sirve para buscar mercados donde funcione.

Antes de empezar

Las mismas condiciones que el lote: el databank de retest exportado y las barras M1 al día. No toca SQX.

Cómo se ejecuta

```
python3 -m studies.transfer.crossmarket.report --project Trade_NIKKEI225_M30_volatilidadL \
    --databank "Retest Markets - Family" --asset NIKKEI225 --day 2026-10-04 \
    --strategy "Strategy 14.3.479"
```

flag	obligatorio	qué hace
`--project`, `--databank`, `--asset`	sí	el export y el activo base
`--day`	no (hoy)	fecha del export
`--strategy`	sí, aquí	la estrategia que se estudia entera
`--only`	no	con `--strategy`, un solo mercado (feed `XAUUSD_M1`, etc.)
`--set travel.draws=500`	no	menos corridas al azar para una prueba

Qué produce

Una página `crossmarket/estrategias/Strategy <id>.html` y su `.json` junto al informe del export (`AlgoData/reports/<proyecto>/<databank>/<día>/crossmarket/`). Es lo que pinta la pestaña **Cross Market** de la ventana: la misma salida, sin recalcular.

Cómo se lee el resultado

La pestaña muestra los nueve hechos P1-P9 de la estrategia (lectura de cada uno en `07-otros-mercados-y-timeframes.pdf › 39-crossmarket-lote`) y la z mercado a mercado: un mercado muy negativo es un mercado en contra (P8); una z total que cae al quitar un solo mercado es evidencia que cuelga de él (P5).

Captura pendiente: la herramienta de capturas no puede disparar esa pestaña con un proyecto elegido, así que no se pone una imagen inventada. Cuando exista, se añadirá aquí.

Un ejemplo completo

La estrategia 3.11.426 de las diez de referencia: z conjunta 2,93, z en V2 2,32, quitando su mejor mercado (USDJPY) 2,39, y 9 de 10 mercados con la misma dirección. Es la que mejor viaja de las diez; aun así queda en Limbo mientras P7 no tenga dato.

Qué NO te dice

Que vaya a funcionar en vivo en esos mercados, ni cuál elegir: sólo si el momento de entrada importa más que el azar. Una sola estrategia aislada tiene mucho ruido; léela junto al resto del lote.

Si algo falla

Si el nombre no coincide con el de la tabla del export (`--strategy` se escribe tal cual: `Strategy 3.11.426`), el comando no encuentra la estrategia. Si un mercado sale «no válido», mira `07-otros-mercados-y-timeframes.pdf › 39-crossmarket-lote`.

5. Paso 10 — ¿la lógica viaja a otros mercados?

> Este capítulo explica **qué pregunta el paso 10 y cómo se juzga**. El comando de una estrategia está en `07-otros-mercados-y-timeframes.pdf › 30-crossmarket`; el del export entero, los archivos y las reglas P1-P9, en `07-otros-mercados-y-timeframes.pdf › 39-crossmarket-lote`. No confundir con el *Retest de Monte Carlo* (`08-montecarlo.pdf › 11-retest-mc`).

Qué pregunta responde

Una estrategia que gana en su activo puede haber captado algo del mercado o simplemente haber estado larga mientras subía. El paso 10 la lleva a otros mercados (los de `assets/_markets.yaml`, cada uno con sus costes) y pregunta: **¿acierta cuándo entra, o le habría dado igual entrar otra semana?**

La idea, sin fórmulas:

1. Se toma la corrida **real**: las mismas operaciones, en cada mercado, medidas en **R** (1R = el riesgo de la doctrina, `Amount` de `assets/_build.yaml`) y con el tamaño que la doctrina pondría en ese momento (según el ATR de la barra anterior a la entrada).
2. Se fabrican miles de corridas **al azar**: las mismas operaciones desplazadas un número entero de **semanas dentro de cada año natural**. Entera semanas conserva el día de la semana y la hora; quedarse dentro del año conserva el régimen de mercado. Lo único que se rompe es el momento exacto de la entrada.
3. El desplazamiento **es el mismo en todos los mercados** (se sortea por año, no por mercado): si 2013 se corre tres semanas, se corre tres semanas en todos. Así las corridas al azar conservan la correlación entre mercados y el total no se infla contando varias veces lo mismo.
4. La **z** dice cuántas desviaciones se separa la real de las corridas al azar. Z alta: la entrada importa. Z cerca de 0: da igual cuándo entre.

Se mira en tres **ventanas**, cortadas en las fechas del activo principal (derivadas, nunca escritas a mano): **V0** = los años anteriores a su `build`, **V1** = su `build` y **V2** = su `oos1` (los años que no vio al construirse). P4 lee V2.

Por qué la z va en R bruto. El coste en R es coste / ATR a la entrada: para entradas condicionadas por el ATR no se cancela contra las copias desplazadas y fabrica una z falsa (medido: de +0,98 neto a +0,30 bruto en una población de ATR alto). Por eso `travel.judge_on: gross`. La ventaja por operación (`e`) y lo que se construye con ella sí van en neto.

Cuándo lo usas, y cuándo no

Cuando una población ha pasado la puerta del paso 8, en el paso 10 del workflow (`bin/workflow.sh` lo corre solo). **No** lo uses para buscar en qué mercado funciona una estrategia: los mercados están fijados de antemano y elegirlos después de ver el resultado es seleccionar. Y no sirve para estrategias cuyo mercado base no tiene tramo `build` / `oos1` declarado.

Antes de empezar

El databank de retest corrido y exportado (`python3 -m sqx.export.export_retest --project P --databank D`) y las barras M1 de los mercados al día (`python3 -m sqx.export.sync_bars`). No toca SQX, así que se puede lanzar con la GUI abierta.

Cómo se ejecuta

Un solo comando (detallado en `07-otros-mercados-y-timeframes.pdf › 39-crossmarket-lote`):

```
python3 -m studies.transfer.crossmarket.report --project P --databank D --asset A
```

Cuesta poco: las diez estrategias de referencia, once mercados y 5.000 corridas al azar, tardan **unos 21 segundos y 755 MB** en un solo proceso. No hay que esperar a SQX ni usar la regla de espera del conductor.

Qué produce

`crossmarket/verdict.csv` (una fila por estrategia), `markets.parquet` (estrategia × mercado) y `crossmarket.json`. Los detalla `07-otros-mercados-y-timeframes.pdf › 39-crossmarket-lote`.

Cómo se lee el resultado

El estudio **no decide**: escribe nueve hechos por estrategia (P1-P9) y el juez de `pipeline/autopilot/criteria.yaml` (paso 10) los corta en Pasa / Limbo / Falla. Una z de 2,33 o más en el conjunto es lo que se pide para Pasa (P3). Las líneas numéricas son **provisionales y sin calibrar**; mientras el mono del paso 8 no dé `e_principal`, P7 queda en Limbo y **ninguna estrategia puede pasar el paso 10**.

Un ejemplo completo

Diez estrategias de `Trade_NIKKEI225_M30_volatilidadL`, once mercados (uno, XCUUSD, queda fuera por precios que no cuadran con el backtest). Valores de z conjunta (`z_conj`), de la salida real `AlgoData/scratch/paso10_travel/2026-10-07-bruto/verdict.csv`:

estrategia	z_conj	z en V2 (oos1 del principal)	z quitando el mejor mercado
14.3.479	3,51	0,54	2,72
3.9.284	3,64	1,82	2,87
3.11.426	2,93	2,32	2,39
4.13.338	2,76	2,19	2,04
10.11.484	2,45	0,05	1,73
19.2.363	0,70	0,44	0,26
13.3.390	0,53	−0,69	0,03
15.9.352	−0,41	−0,74	−0,77
18.5.355	−0,73	1,36	−1,23
16.5.352	−0,88	−0,33	−1,10

Media de la z conjunta: +1,45. Con el juez aplicado a estas diez (corte del 2026-10-07): **0 Pasa, 5 Limbo, 5 Falla**. Una parte de esa z positiva es de las estrategias (se concentra en los años del build) y otra es ruido del aparato; el aparato corregido deja un sesgo de uniformes de +0,07 ± 0,07.

Qué NO te dice

- Que la estrategia **gane** en esos mercados: solo que su momento de entrada importa más que el azar. Un mercado con deriva fuerte no la convierte en señal.
- Qué mercado «funciona»: la z de cada mercado es una anotación, no un filtro.
- Nada con las líneas actuales: están sin calibrar contra una población nula ni una con ventaja plantada.

Si algo falla

- **Un mercado «no válido»** (`fill_mismatch`, valor del punto distinto al de la ficha): sale de todas las pruebas; si pasa del 25 % de la lista, la estrategia queda en Limbo.
- **Un par (mercado, ventana) «sin dato»** por huecos de datos: se lee como sin dato, nunca como 0.
- **Estrategias fuera de la ventana propia de su mercado** no se leen; el comando lo avisa.

39. Paso 10 de la población entera — ¿viaja la lógica?, un veredicto por estrategia

> La idea y cómo se juzga: `07-otros-mercados-y-timeframes.pdf › 05-retest-mercados`. Una sola estrategia: `07-otros-mercados-y-timeframes.pdf › 30-crossmarket`.

Qué pregunta responde

Para todas las estrategias de un export a la vez: ¿el momento de entrada importa más que el azar en otros mercados? Escribe nueve hechos por estrategia (P1-P9); el juez de `pipeline/autopilot/criteria.yaml` (paso 10) los corta en Pasa / Limbo / Falla.

Cuándo lo usas, y cuándo no

Después del paso 8, con el databank de retest exportado; `bin/workflow.sh` lo corre solo. No sirve para explicar por qué una estrategia falla en un mercado (eso es `07-otros-mercados-y-timeframes.pdf › 30-crossmarket`) ni para elegir mercados.

Antes de empezar

1. El databank corrido y exportado: `python3 -m sqx.export.export_retest --project P --databank D`.
2. Las barras M1 de todos los mercados al día: `python3 -m sqx.export.sync_bars`.
3. El activo declarado en `assets/_markets.yaml` (de ahí sale la lista de mercados).

No toca SQX ni usa más CPU que un proceso.

Cómo se ejecuta

```
python3 -m studies.transfer.crossmarket.report --project Trade_NIKKEI225_M30_volatilidadL \
    --databank "Retest Markets - Family" --asset NIKKEI225 --day 2026-10-04 \
    --set travel.draws=500
```

flag	obligatorio	qué hace
`--project`, `--databank`, `--asset`	sí	el export y el activo base
`--day`	no (hoy)	fecha del export, AAAA-MM-DD
`--set travel.draws=500`	no	cambia un parámetro de `config.yaml` para esta corrida; `travel.draws=500` es la prueba rápida (por defecto 5.000)
`--strategy S`, `--only M`	no	una estrategia entera (`07-otros-mercados-y-timeframes.pdf › 30-crossmarket`)

Tarda unos 21 s y 755 MB con diez estrategias, once mercados y 5.000 corridas.

Qué produce

En `AlgoData/reports/<proyecto>/<databank>/<día>/crossmarket/`:

- `verdict.csv`: una fila por estrategia, con los hechos de P1-P9. Una celda vacía es «sin dato», nunca 0.
- `markets.parquet`: estrategia × mercado (z, p, ventaja por operación `e` en R, operaciones, z de frecuencia, si el mercado es válido y por qué no).
- `crossmarket.json` (y `.md`, `.html`): el resumen que lee la ventana, con la serie anual de P9.

Sobrescribe el informe del mismo día.

Cómo se lee el resultado

Las diez de referencia (`Trade_NIKKEI225_M30_volatilidadL`, once mercados, uno inválido), de `AlgoData/scratch/paso10_travel/2026-10-07-bruto/verdict.csv`, ordenadas por z conjunta:

estrategia	fijas	freq_off	z_conj	z_v2	z_sin_mercado	p_amplitud	worse	z_beta	e_end	op.
3.9.284	0	0	3,64	1,82	2,87	0,08	0	0,93	0,072	4482
14.3.479	0	0	3,51	0,54	2,72	0,08	0	-0,92	0,020	3671
3.11.426	0	0	2,93	2,32	2,39	0,02	0	1,01	0,092	2551
4.13.338	0	0	2,76	2,19	2,04	0,10	0	1,53	0,103	3900
10.11.484	0	0	2,45	0,05	1,73	0,21	0	0,01	0,041	3197
19.2.363	0	5	0,70	0,44	0,26	0,24	0	-0,14	0,020	1554
13.3.390	0	3	0,53	-0,69	0,03	0,40	0	-0,99	-0,020	1562
15.9.352	0	0	-0,41	-0,74	-0,77	0,39	1	0,53	0,003	5048
18.5.355	0	0	-0,73	1,36	-1,23	0,80	1	2,64	0,074	3269
16.5.352	0	0	-0,88	-0,33	-1,10	0,79	0	0,34	-0,003	4850

(`fijas` = P1, parámetros de distancia fija; `worse` = mercados en contra, P8; `op.` = operaciones.) El informe de la misma población con el corte aplicado (`2026-10-07-corte/crossmarket/`) lista además los mercados con control previo: XCUUSD queda **no válido** (`fill_mismatch 0.581 ATR`: sus velas no son las del backtest) y diez pares (mercado, año de V1) quedan **sin dato** por huecos de datos (p. ej. HK50 2012: 90 % de las entradas se irían a una barra posterior).

Las nueve pruebas, en palabras. Pasa / Limbo / Falla. Las líneas numéricas son **PROVISIONALES y SIN CALIBRAR** (`criteria.yaml`, comentario del paso 10); léelas como una propuesta, no como un umbral demostrado.

prueba	qué comprueba	cortes
P1	que las reglas no usen distancias fijas en puntos o pips, que no viajan de un mercado a otro	0 pasa; si no, Limbo
P2	que la frecuencia de operar en cada mercado no sea rara frente a la de casa	0 mercados raros pasa; 1-2 Limbo; 3 o más (o más del 25 %) Falla
P3	z conjunta frente al azar, ventana completa	z ≥ 2,33 pasa; ≥ 0 Limbo; negativa Falla
P4	que siga en V2, los años que no vio	z ≥ 0 pasa; > −1,65 Limbo; si no, Falla
P5	que quitando cada mercado la evidencia siga ahí (no cuelga de uno)	z ≥ 1,65 pasa; si no Limbo; nunca Falla
P6	amplitud: la dirección se repite en más mercados de lo que daría la suerte	p ≤ 0,10 pasa; si no Limbo; nunca Falla
P7	rendimiento mínimo en la mitad de los mercados frente a casa	≥ 0,5 pasa; > 0 Limbo; **sin dato Limbo**
P8	mercados en contra (z ≤ −1,65)	hasta 1 pasa; 2 Limbo; 3 o más (o más del 25 %) Falla
P9	degradación en el tiempo: que la ventaja no se esté apagando	pasa si z_beta > −1,65, o > −2,33 con e_end > 0; Falla si ≤ −2,33 y e_end ≤ 0; lo demás Limbo

P7 está **sin dato** hasta que el mono del paso 8 entregue `e_principal`: hoy ninguna estrategia puede dar Pasa en el paso 10 (las diez de referencia dieron 0 Pasa, 5 Limbo, 5 Falla). Además, un mercado inválido sale de todas las pruebas, y si son más del 25 % de la lista la estrategia queda en Limbo.

Los parámetros y dónde se editan

Todo lo que define un cálculo está en `studies/transfer/crossmarket/config.yaml`, bajo `travel:`. Se edita a mano en ese archivo, con `--set travel.<clave>=<valor>` para una corrida, o en el **cajón de configuración** del estudio en la ventana (cada parámetro con su frase al pasar el ratón: `studies/transfer/crossmarket/tooltips.py`). Los cortes Pasa/Limbo/Falla, en cambio, están en `criteria.yaml` (zona «Criterios» de la ventana): cambian el veredicto, no los hechos.

parámetro	valor	qué es
`draws`	5000	corridas al azar; el p más pequeño posible es 1/(draws+1)
`seed`	20261007	semilla del sorteo conjunto
`judge_on`	gross	`gross` o `net`: sobre qué se leen las z (la ventaja `e` va siempre en neto)
`shift_unit`	year	`year` o `semester`: el bloque dentro del que se desplaza
`shift_step`	week	`week` o `day`: en qué se cuenta el desplazamiento
`min_trades_market`	30	por debajo, el mercado suma pero no cuenta a favor ni en contra
`worse_z`	−1,65	la z que marca un mercado en contra (P8)
`frequency`	z 3,5; φ 1,5–3,5	control de P2
`validity`	punto 0,02; plano 0,02; desplazadas 0,20; error de precio 0,25	controles de datos por mercado
`degradation`	30 op./año; 4 años	P9: años con menos operaciones se agrupan

Un ejemplo completo

Es el comando de arriba con el `--set` de prueba; la salida es la tabla de la sección anterior y las líneas `10 estrategias medidas -> .../crossmarket/verdict.csv`. De las diez, 14.3.479, 3.9.284, 3.11.426, 4.13.338 y 10.11.484 superan z 2,33 en P3, pero 14.3.479 y 10.11.484 apenas puntúan en V2 (0,54 y 0,05): el caso típico de una ventaja concentrada en los años del build.

Qué NO te dice

Que la estrategia gane dinero en esos mercados, ni qué mercado elegir; ni nada firme con las líneas actuales, que no se han calibrado contra una población nula y otra con ventaja plantada.

Si algo falla

- «N operaciones fuera de la ventana propia de su mercado»: se ignoran, es un aviso.
- Un mercado «no válido» o pares «sin dato»: ver arriba; salen de las pruebas.
- Faltan barras de un mercado: `python3 -m sqx.export.sync_bars --check`.

31. crossTF — ¿el edge aguanta si lo miras en un reloj más lento?

Qué pregunta responde

Coges una estrategia hecha en H1 y la miras en H4. Si lo que gana en H1 es una ventaja de verdad, debería seguir estando ahí cuando solo puedes actuar cada cuatro horas en vez de cada hora. Si desaparece, o era una ventaja de temporización tan fina que no la vas a poder ejecutar, o no era una ventaja.

El módulo hace esa comparación **dos veces, de dos maneras distintas**, porque son dos preguntas diferentes y solo una es un test de robustez:

	qué le haces a la estrategia	**qué significa que falle**
escalada	le divides los períodos entre 4 (una media de 20 horas sigue siendo de 20 horas)	**evidencia en contra de la estrategia**
sin escalar	la dejas igual (una media de 20 velas pasa a ser de 80 horas)	**nada sobre la estrategia** — eso pregunta si el mercado es autosemejante, que es otro tema

Cuándo lo usas, y cuándo no

Sí: sobre un puñado de supervivientes que ya han pasado la puerta OOS. Es un test caro por estrategia y su sitio es el final del embudo, no el principio.

No: sobre una población de miles recién construida. Y **no** como puerta binaria: la recomendación es leerlo como una puntuación más, porque H1 y H4 son la misma serie de precios remuestreada y la evidencia que aporta está muy correlacionada con la que ya te dio la puerta.

No para bajar a D1 desde H1. Se puede fabricar, pero el escalado divide entre 24 y los períodos reales se destruyen: medido sobre las tres madres de 2026-09-22, los parámetros se mueven entre un 140 % y un 586 % y **todos** quedan pegados al mínimo del constructor. Una variante así no es la estrategia escalada, es otra estrategia. D1 solo tiene sentido en la fila sin escalar de la madre.

Antes de empezar

1. Las madres, en una carpeta, como `.sqx`. No se tocan: el módulo solo lee.
2. `python3 -m core.assets XAUUSD` en verde, antes de tocar ninguna tarea de SQX.
3. Las tareas configuradas con `sqx.projects.crosstf`: una por timeframe, cada una con su databank, y todas sin condiciones de aceptación: queremos evidencia, no un filtro de selección.
4. El worker custodio parado antes de arrancar nada, y comprobar que no lo está usando otra sesión (`ls -lt user/projects` y el log).

> En la página de población de la ventana, el selector **«Madre»** elige de qué madre se pintan las curvas de equity (una por madre, `studies/transfer/crossTF/equity.py`).

Desde la ventana (lo normal)

Desde el 29-09 **no hay que hacer nada a mano entre el Cross Market y el Cross TF**. El orden es fijo: Cross Market → Cross TF → MC Retest. Al pulsar ▶ SQX del paso 11 (o «Correr workflow», o «Continuar workflow» sobre el databank de Cross Market), la ventana, con el worker parado:

1. coge las supervivientes que quedan en el databank de Cross Market;
2. las escala a los timeframes de la doctrina — desde H1: **M30, H4 y D1**; desde M30: **M15**, H1 y H4 —;
3. vacía `CrossTF_Input` y copia ahí madres y hermanas;
4. escribe **una tarea por timeframe** (dueño, 2026-09-30): «CrossTF» corre el timeframe de la madre, y «CrossTF M30», «CrossTF H4» y «CrossTF D1» cada uno el suyo, con su propio databank (`CrossTF_M30`, `CrossTF_H4`, `CrossTF_D1`). **D1 corre con el motor de MetaTrader 4**;
5. arranca el worker, corre las cuatro tareas de un tirón, lo para y exporta cada databank.

Por qué una tarea por timeframe y no los bloques del crosscheck. SQX exporta las operaciones de todos los bloques de «Retest on additional markets» mezcladas por hora, y con el mismo símbolo no hay ninguna columna que diga de qué bloque es cada una. El código las separaba por el orden de los tickets, y eso asigna los bloques por **lo rápido que opera cada uno**, no por su timeframe: desde H1, las operaciones de M30 se leían como las de H1. Con una tarea por timeframe cada export es inequívoco (medido: M30 abre a la media hora en el 52 % de las operaciones, H4 solo a las 4, 8, 12, 16 y 20 h, D1 solo a las 00:00).

Por qué D1 con MetaTrader 4. En D1 la estrategia sólo entra a las 00:00, y la sesión de las fondeadas empieza a las 00:05: con el motor de MT5 no hacía ninguna operación. El motor de MT4 no pide sesión. Lo dice `crosstf.engines` en `assets/_build.yaml`.

Y antes del MC Retest (paso 13) copia a `CrossTF_Mothers` las madres que quedan en «CrossTF», sin sus hermanas escaladas: eso es lo que perturba el MC Retest.

Lo mismo a mano, con el worker parado:

```
python3 -m sqx.projects.crosstfload --project Test_USDJPY_donchianUpperCrossUp_H1 --role custodian
python3 -m sqx.projects.crosstfload --project Test_USDJPY_donchianUpperCrossUp_H1 --role custodian --mothers
```

Dos opciones nuevas (2026-10-02), para alimentar el paso desde fuera de la cadena:

flag	qué hace
`--from-folder CARPETA`	toma las madres de esa carpeta (todo `.sqx` que haya debajo) en vez del databank de Cross Market, y vacía antes los databanks de salida. Es lo que usa la calibración de criterios con `calib2/keep/<proyecto>/`
`--targets M30 H4`	escala y reprueba solo esos timeframes de la lista de la doctrina. La calibración de criterios corre **sin D1** («quítalo»): desde H1 todas las hermanas D1 salen pegadas al mínimo. La doctrina y la tarea «CrossTF D1» con MetaTrader 4 siguen como estaban

Salida real (corrida del 30-09, 30 madres):

```
30 madres x 3 timeframes -> 90 hermanas (48 con períodos pegados al mínimo: se leen solo sin escalar)
desde H1: M30, H4, D1 · una tarea por timeframe: H1, M30, H4, D1
120 .sqx en CrossTF_Input (madres + hermanas) · lote .../AlgoData/crosstf/<proyecto>/2026-09-30
«CrossTF M30»: M30 con MetaTrader5 (hedged) -> CrossTF_M30
«CrossTF H4»: H4 con MetaTrader5 (hedged) -> CrossTF_H4
«CrossTF D1»: D1 con MetaTrader4 -> CrossTF_D1
```

Un timeframe más rápido que el de la madre vale (H1 → M30): los períodos se doblan, sin redondeo y sin pegarse al mínimo. D1 desde H1 sale siempre pegado al mínimo, y la calibración de criterios del 2026-10-02 corre sin D1.

Cómo se ejecuta paso a paso

Son tres pasos: fabricar, correr, leer.

1 · Fabricar las hermanas escaladas

```
python3 -m sqx.variants.scale \
  --mothers ~/Desktop/AlgoData/projectsBackup/madres-2026-09-22 \
  --project XAUUSD --source H1 --targets H4
```

flag	obligatorio	qué hace
`--mothers`	sí	carpeta con los `.sqx` originales
`--project`	sí	de qué proyecto vienen; da nombre al árbol de salida
`--out`	no	solo para sacarlo fuera del sitio declarado; normalmente no se usa
`--source`	no (H1)	el timeframe en el que se construyeron las madres
`--targets`	no	a qué timeframes se escalan; por defecto los de la doctrina según `--source`: desde M30, M15, H1 y H4; desde H1, **M30, H4 y D1**
`--symbol`	no	el activo cuyas velas dan el cociente de ATR `c` con que se dividen los coeficientes por vela; por defecto, el de la primera madre

Qué se escala (desde el 2026-10-02). Todo parámetro que sea una longitud en velas, y nada más: lo que el propio SQX marca como periodo en la estrategia (los tres del MACD, el slowing del estocástico, las longitudes de WaveTrend, QQE o BH Ergodic, que antes se quedaban sin tocar en el 65 % de las estrategias), la salida por número de velas, y en los bloques custom lo que diga la tabla `sqx/variants/lengths.yaml`, bloque a bloque. Niveles, multiplicadores, desviaciones, shifts y selectores no se tocan nunca. Un estocástico (5, 3, 3) de H1 pasa a (10, 6, 6) en M30. Un bloque custom que la tabla no conoce se deja como está y su hermana sale `unusable`.

Segundos. No toca SQX, se puede lanzar con la GUI abierta.

2 · Configurar las tareas, correrlas y exportarlas

```
python3 -m core.assets USDJPY                              # regla dura 5, bloqueante
python3 -m sqx.projects.crosstf USDJPY \
    --cfx ~/Desktop/SQX_w2/user/projects/Test_USDJPY_donchianUpperCrossUp_H1/project.cfx
```

flag	obligatorio	qué hace
`--cfx`	sí	el proyecto **custom**, nunca el `Retester` de serie (regla dura 10)
`--day`	no	el día del lote bajo el que guarda `blocks.json`; por defecto hoy

Pone «CrossTF» en el tramo `oos1` (desde el 2026-10-02 el paso lee **solo oos1**, a costes de oos1 y con la precisión «timeframe elegido»; antes era `build..oos1`), apaga su crosscheck y **todas** sus condiciones de aceptación (esto mide, no filtra: el corte se hace en Python y se aplica con `/curate`), y copia de ella una tarea por cada timeframe extra, cada una con su databank y su motor. Deja escrito en `blocks.json`, junto al lote, qué timeframe y qué databank es cada bloque: el estudio lee eso, nunca la doctrina de hoy. Imprime una línea por tarea:

```
CrossTF: H1, costes de USDJPY a `build..oos1`
«CrossTF M30»: M30 con MetaTrader5 (hedged) -> CrossTF_M30
«CrossTF H4»: H4 con MetaTrader5 (hedged) -> CrossTF_H4
«CrossTF D1»: D1 con MetaTrader4 -> CrossTF_D1
```

Luego cargas las madres *y* las hermanas en `CrossTF_Input` con el worker parado (`sqx.projects.crosstfload`, que ya llama a esto), corres las cuatro tareas y, al pararse el worker, cada databank se exporta solo.

3 · Leer

```
python3 -m studies.transfer.crossTF.report --project Test_USDJPY_donchianUpperCrossUp_H1 \
    --asset USDJPY --day 2026-09-30 --fabricated 2026-09-30
# la misma firma que los demás estudios: proyecto, databank (CrossTF por defecto), activo y
día.
# El export de CrossTF y los de CrossTF_<TF> (uno por timeframe, al lado) y el scaling.parquet
# salen de ahí; el feed, del activo en assets/.
```

Qué produce

Todo en `~/Desktop/AlgoData/crosstf/<proyecto>/<fecha>/`, fechado e inmutable (regla dura 7: los datos pesados nunca en el repo). **Su subcarpeta `sqx/` es la carga del databank**, tal cual (desde el 27-09; antes era la carpeta entera y SQX se quejaba en el log de `scaling.parquet`):

- `sqx/<madre>_ScaledH4.sqx` — una por madre y timeframe destino. 14 KB cada una.
- `sqx/<madre>.sqx` — **las madres, copiadas ahí al lado sin tocar**, porque el estudio necesita también sus celdas `baseline` y `control`. Así se carga una carpeta y no dos.
- `scaling.parquet`, fuera de `sqx/` — qué parámetro se movió y a cuánto, por hermana. **Es el registro de lo que se hizo**: sin él, un `.sqx` escalado es indistinguible de uno cualquiera.
- `--out` de `report.py`, opcional: el panel entero en Parquet.

Nada se sobrescribe salvo que apuntes dos veces a la misma carpeta.

Cómo se lee el resultado

El paso 1 imprime lo que ha movido:

```
3 mothers x 2 targets -> 6 siblings
                      name  ratio  n_scaled  max_rounding_shift  clamped
Strategy 10.13.25_ScaledH4    4.0         2            0.090909    False
Strategy 10.13.30_ScaledH4    4.0         6            0.200000    False
Strategy 29.30.36_ScaledH4    4.0         4            0.142857    False
Strategy 10.13.25_ScaledD1   24.0         2            1.400000     True
Strategy 10.13.30_ScaledD1   24.0         6            4.333333     True
Strategy 29.30.36_ScaledD1   24.0         4            5.857143     True

int parameters left untouched (10) -- review the whitelist:
  BBBarOpensShift1, CBlock_WssShfInt21, CloseDShift1, IchimokuSnkSpnCrsBshSgnStr1, ...
```

Dos columnas que se apuntan y se enseñan, y que **ya no descartan a ninguna hermana** (decisión tuya del 2026-10-02: «Redondea donde puedas, ya está»). Cada longitud se redondea al entero más cercano, con un mínimo de 2, y la hermana se lee como cualquier otra:

- `max_rounding_shift` — cuánto se desvió el parámetro peor por culpa del redondeo. Un período de 9 entre 4 es 2,25 y hay que escribir 2: eso es un 11 % de cambio **encima** del cambio de timeframe. **Menos es mejor**; queda como dato para medir después si las hermanas muy redondeadas se comportan distinto.
- `clamped` — un período que habría quedado por debajo de 2 y se ha pegado al mínimo. Queda apuntado; la fila se lee igual.

La última línea lista los parámetros enteros que el escalado **no** ha tocado. Están ahí para revisarlos, no porque haya fallado nada: la mayoría son shifts y selectores categóricos, que no deben escalarse.

El paso 3 imprime tres bloques. Primero la madre en cada timeframe sin escalar, **sin veredicto**, porque esa fila no juzga a la estrategia. Después las hermanas escaladas, cada una con su lectura:

lectura	qué pasó
`survives`	gana a su propio nulo de temporización en oos1 **y además** añade algo a lo que ya enseñó su madre (`z_new`)
`inherited`	gana dinero en oos1, pero no más de lo que el resultado de su madre ya predice
`fails`	pierde dinero en oos1
`no_data`	**sin datos**: menos de 30 operaciones en oos1, en la hermana o en su madre. No se juzga
`silent`	no hizo ninguna operación en oos1 en ese timeframe. No se juzga
`unusable`	lleva un bloque custom con una longitud sin verificar: no se pudo escalar. No se juzga. (El redondeo y el tope ya no descartan a nadie)

⚠ **Esta lectura es una propuesta de un agente del 2026-10-02, pendiente de tu aprobación.** Lo que sí decidiste ese día es que el paso lee solo oos1. Lo que la propuesta añade:

- **Solo oos1.** La estadística, el nulo y la p de cada celda son de las operaciones de oos1. La madre se eligió sobre el tramo de construcción: medida ahí salía siempre «significativa».
- `z`: a cuántas desviaciones del nulo queda la celda. Contra una estrategia sin ventaja y sin costes, 1,65 lo supera el 5 % y 2,33 el 1 %.
- **`z_new`: lo que la hermana añade a su madre.** Una hermana vuelve a leer el mismo activo en los mismos años. Si la madre pasó por suerte, su hermana «también pasa» entre el 14 % y el 40 % de las veces, no el 5 %. `z_new` descuenta lo que el resultado de la madre ya explica, usando lo parecidas que son las dos (la correlación de sus resultados mensuales): con él, el 5 % vuelve a ser el 5 %.
- **La suma por madre** (`combined_z_new`): lo que añaden juntas sus hermanas utilizables. Es el número sobre el que se escribirán las reglas del paso 12.
- **El control se enseña y no se juzga.** Era la lectura `control_failed`: se llevaba 29 de 46 hermanas reales y se invertía cuando la madre perdía dinero. La hermana con los periodos divididos entre cuatro, corrida en las velas de la madre, es otra estrategia, no un control.
- **Semilla.** `--seed N` repite los nulos exactamente; sin ella se sortea una nueva, que queda apuntada en el manifest (`nulls_seed`) y en la última línea que imprime el comando.

Un ejemplo completo

Fabricación real sobre las tres madres, mirando una por dentro antes y después:

```
MADRE H1  (7 miembros, 101.9 KB)          HIJA H4  (5 miembros, 14.4 KB)
   CBlock_KSBreakOPrd11        = 20          CBlock_KSBreakOPrd11        = 5
   CBlock_KSBreakOPrd21        = 50          CBlock_KSBreakOPrd21        = 12
   IchimokuSnkSpnCrsBshTnkPrd1 = 9           IchimokuSnkSpnCrsBshTnkPrd1 = 2
   IchimokuSnkSpnCrsBshKjnPrd1 = 26          IchimokuSnkSpnCrsBshKjnPrd1 = 6
   IchimokuSnkSpnCrsBshSnkPrd1 = 36          IchimokuSnkSpnCrsBshSnkPrd1 = 9
   IchimokuSnkSpnCrsBshSgnStr1 = 1           IchimokuSnkSpnCrsBshSgnStr1 = 1   <- intacto
   IchimokuSnkSpnCrsBshShf1    = 1           IchimokuSnkSpnCrsBshShf1    = 1   <- intacto
   ExitAfterBars1              = 10          ExitAfterBars1              = 2
   fingerprint = SI                          fingerprint = no, stamp = CTF001H4
```

Los períodos bajan; el shift y el selector de fuerza de señal se quedan. El `<Fingerprint>` heredado desaparece — si se quedara, el databank podría fundir todas las hermanas en una.

Los pasos 11.5 y 11.7 y la lectura nueva (encargo 47, construida AL LADO)

Desde el 07-10 el paso 11 se parte: **11** corre en oos1; **11.5** es el estudio nuevo en Python, fase 1 (T0, T2a, T3 con lo que hay en oos1); **11.7** corre en SQX en la ventana `build` las tareas gemelas `CrossTF <TF> build` (databanks `CrossTF_<TF>_build`), sólo para las madres que siguen, sin rellenar `CrossTF_Input`; **12** vuelve a leer con el build (T1, T2b). Un proyecto sin las tareas gemelas se salta el 11.7 con una línea. El estudio nuevo pregunta si la ventaja es un **pico aislado en su timeframe**: madre y hermanas pasan por las mismas corridas al azar (un desplazamiento por año natural, igual para todas) y salen `tf_siblings.parquet` y `tf_mothers.csv` junto a lo viejo. **Todavía no manda:** la regla viva sigue siendo `crossTF.combined.z_new` hasta que una corrida real lo valide y digas «corta». Tampoco se escala ya «sólo

periodos»: los coeficientes que multiplican una medida por vela (múltiplos de ATR) se dividen por `c`, el cociente medido de ATR entre el timeframe de la hermana y el de la madre (NIKKEI225 desde M30: M15 0,704 · H1 1,419 · H4 2,833).

Qué NO te dice

- **No te dice que la estrategia sea buena.** Dice si su ventaja sobrevive a mirarla más despacio. Una estrategia mala puede pasar el test perfectamente.
- **La celda H4 no es «la misma estrategia, reloj más lento».** Por decisión del dueño (2026-09-23) solo se escalan períodos y salidas por barras. Los `StopLossCoef` y `ProfitTargetCoef` se escriben tal cual, y como multiplican un rango que crece al agregar velas, **el stop en H4 es físicamente el doble de ancho en precio**. Es una propiedad declarada de la comparación, no un fallo — pero no la describas como limpia. (Desde el 07-10 los coeficientes por vela se dividen por `c`; ver arriba.)
- **H1 y H4 no son muestras independientes.** Una vela H4 está hecha de las mismas cuatro velas H1. Esto aporta mucha menos información que probar en otro mercado, y por eso el veredicto se apoya en el nulo de cada timeframe y no en «¿es rentable?».
- **No dice nada sobre datos no vistos.** Si el retest cae dentro de la ventana en la que se seleccionó la estrategia, sigue siendo ventana seleccionada.

Si algo falla

- **«CrossTF_D1 no tiene export».** Cada timeframe se exporta al parar el worker; si falló (el conductor estaba ocupado), pulsa ↻ en su databank o corre `python3 -m ui.daemon.loader.afterrun --role custodian`.
- **Una corrida anterior al 30-09 da números raros.** Entonces los timeframes eran bloques del crosscheck y se separaban por el orden de los tickets, que los confunde (M30 se leía como H1). El estudio lo avisa: esa corrida no es fiable; repítela.
- **Sale `unusable`.** Solo pasa cuando la estrategia lleva un bloque custom que la tabla `sqx/variants/lengths.yaml` no conoce o no ha podido verificar: hay que añadirlo ahí.